

Advanced Machine Learning

Lecture 1: Introduction & Basics

Jens-Peter M. Zemke
zemke@tuhh.de

Institut für Mathematik
Lehrstuhl Numerische Mathematik
Technische Universität Hamburg

15 October 2024



Outline

Introduction

- What do you expect?
- Biological inspiration

Basics

- Support vector machines
- Activation functions
- Artificial neural networks
- Universal approximation theorem

Outline

Introduction

What do you expect?
Biological inspiration

Basics

Support vector machines
Activation functions
Artificial neural networks
Universal approximation theorem

Advanced Machine Learning

You are here to attend the lecture

Advanced Machine Learning.

Advanced Machine Learning

You are here to attend the lecture

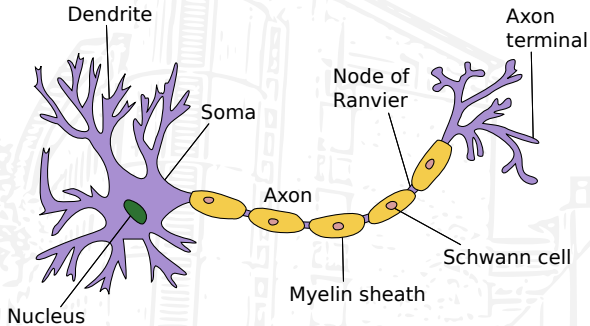
Advanced Machine Learning.

The most advanced machine learning models are **neural networks**, I am a **mathematician**, therefore the

Mathematics of **Neural Networks**

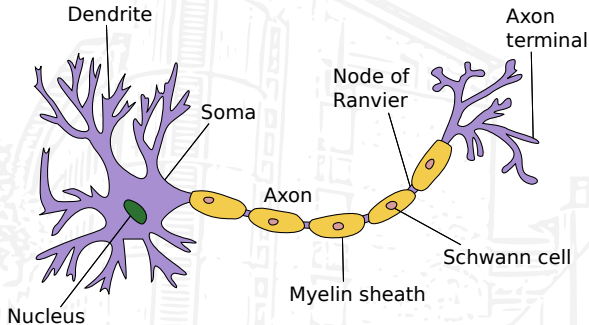
is our main concern, heavily focusing on the **efficient implementation** of the networks.

A “real” neuron



By Quasar Jarosz at English Wikipedia, CC BY-SA 3.0,
<https://commons.wikimedia.org/w/index.php?curid=7616130>

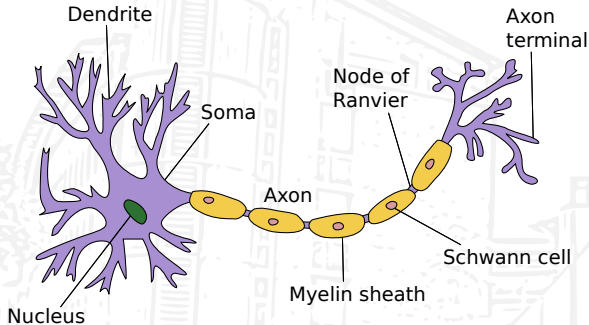
A “real” neuron



Biological neuron: cell that fires an electrical impulse along the **axon** in case the incoming electrical impulses along the **dendrites** reach a certain threshold. The **axon terminals** are connected to the dendrites of other neurons or an output. Neurons that convey information from tissues and organs are called **afferent neurons**, neurons that transmit signals to the effector cells are termed **efferent neurons**. Neurons that connect neurons are termed **interneurons**.

By Quasar Jarosz at English Wikipedia, CC BY-SA 3.0,
<https://commons.wikimedia.org/w/index.php?curid=7616130>

A “real” neuron



By Quasar Jarosz at English Wikipedia, CC BY-SA 3.0,
<https://commons.wikimedia.org/w/index.php?curid=7616130>

Biological neuron: cell that fires an electrical impulse along the **axon** in case the incoming electrical impulses along the **dendrites** reach a certain threshold. The **axon terminals** are connected to the dendrites of other neurons or an output. Neurons that convey information from tissues and organs are called **afferent neurons**, neurons that transmit signals to the effector cells are termed **efferent neurons**. Neurons that connect neurons are termed **interneurons**.

Mathematical neuron: uses (weighted) sum of input to decide, depending on a **threshold**, to **output** either **zero** (no activation) or **one** (activation).

Outline

Introduction

What do you expect?
Biological inspiration

Basics

Support vector machines
Activation functions
Artificial neural networks
Universal approximation theorem

Neural Networks

Artificial Neural Networks (ANN): have been around since 1943, gained a lot of attention in recent years. **Online textbooks:** Nielsen [5], Goodfellow, Bengio, and Courville [1], and Zhang et al. [8], **mathematical treatment:** Higham and Higham [2], Roberts and Yaida [6].

Neural Networks

Artificial Neural Networks (ANN): have been around since 1943, gained a lot of attention in recent years. **Online textbooks**: Nielsen [5], Goodfellow, Bengio, and Courville [1], and Zhang et al. [8], **mathematical treatment**: Higham and Higham [2], Roberts and Yaida [6].

Our approach: **affine functions**, not biology. The affine map

$$L : \mathbb{R}^n \rightarrow \mathbb{R}^m, \quad L(\mathbf{x}) = \mathbf{A}\mathbf{x} - \mathbf{b}$$

is a combination of scalar affine maps

$$\ell_j : \mathbb{R}^n \rightarrow \mathbb{R}, \quad \ell_j(\mathbf{x}) = \mathbf{w}_j^\top \mathbf{x} - b_j = \mathbf{x}^\top \mathbf{w}_j - b_j, \quad j = 1, \dots, m,$$

vector $\mathbf{w}_j^\top := \mathbf{e}_j^\top \mathbf{A}$ comprises j th row of $\mathbf{A}$.

Neural Networks

Artificial Neural Networks (ANN): have been around since 1943, gained a lot of attention in recent years. **Online textbooks**: Nielsen [5], Goodfellow, Bengio, and Courville [1], and Zhang et al. [8], **mathematical treatment**: Higham and Higham [2], Roberts and Yaida [6].

Our approach: **affine functions**, not biology. The affine map

$$L : \mathbb{R}^n \rightarrow \mathbb{R}^m, \quad L(\mathbf{x}) = \mathbf{A}\mathbf{x} - \mathbf{b}$$

is a combination of scalar affine maps

$$\ell_j : \mathbb{R}^n \rightarrow \mathbb{R}, \quad \ell_j(\mathbf{x}) = \mathbf{w}_j^T \mathbf{x} - b_j = \mathbf{x}^T \mathbf{w}_j - b_j, \quad j = 1, \dots, m,$$

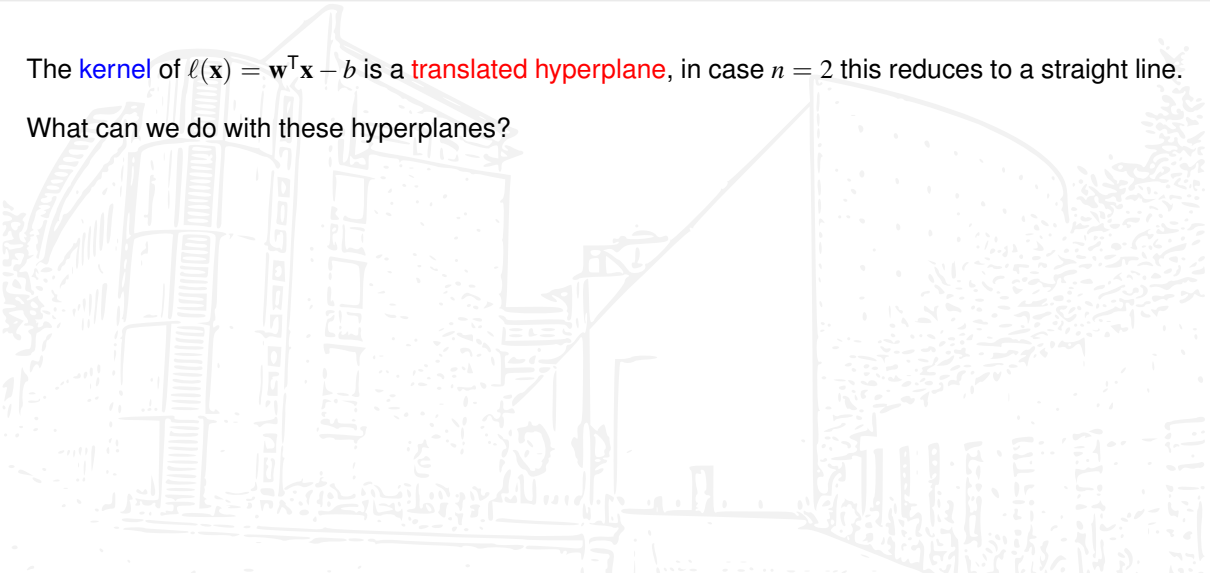
vector $\mathbf{w}_j^T := \mathbf{e}_j^T \mathbf{A}$ comprises j th row of $\mathbf{A}$.

Attention: both notations $\mathbf{w}^T \mathbf{x}$ and $\mathbf{x}^T \mathbf{w}$ are used in the sequel and the sign of $\mathbf{b}$ will change.

Affine maps & hyperplanes

The **kernel** of $\ell(\mathbf{x}) = \mathbf{w}^T \mathbf{x} - b$ is a **translated hyperplane**, in case $n = 2$ this reduces to a straight line.

What can we do with these hyperplanes?



Affine maps & hyperplanes

The **kernel** of $\ell(\mathbf{x}) = \mathbf{w}^T \mathbf{x} - b$ is a **translated hyperplane**, in case $n = 2$ this reduces to a straight line.

What can we do with these hyperplanes? Describe

► solutions of **linear systems**:

$$\mathbf{Ax} = \mathbf{b} \quad \Leftrightarrow \quad \mathbf{Ax} - \mathbf{b} = \mathbf{0} \quad \Leftrightarrow \quad \mathbf{Ax} - \mathbf{b} \leq \mathbf{0} \quad \wedge \quad \mathbf{Ax} - \mathbf{b} \geq \mathbf{0}$$

Affine maps & hyperplanes

The **kernel** of $\ell(\mathbf{x}) = \mathbf{w}^T \mathbf{x} - b$ is a **translated hyperplane**, in case $n = 2$ this reduces to a straight line.

What can we do with these hyperplanes? Describe

- ▶ solutions of **linear systems**:

$$\mathbf{Ax} = \mathbf{b} \Leftrightarrow \mathbf{Ax} - \mathbf{b} = \mathbf{0} \Leftrightarrow \mathbf{Ax} - \mathbf{b} \leq \mathbf{0} \wedge \mathbf{Ax} - \mathbf{b} \geq \mathbf{0}$$

- ▶ solutions of **least-squares problems**:

$$\min \|\mathbf{Ax} - \mathbf{b}\|_2 \Leftrightarrow \min \|\mathbf{r}\|_2 \wedge \mathbf{r} := \mathbf{b} - \mathbf{Ax}$$

Affine maps & hyperplanes

The **kernel** of $\ell(\mathbf{x}) = \mathbf{w}^T \mathbf{x} - b$ is a **translated hyperplane**, in case $n = 2$ this reduces to a straight line.

What can we do with these hyperplanes? Describe

- ▶ solutions of **linear systems**:

$$\mathbf{Ax} = \mathbf{b} \Leftrightarrow \mathbf{Ax} - \mathbf{b} = \mathbf{0} \Leftrightarrow \mathbf{Ax} - \mathbf{b} \leq \mathbf{0} \wedge \mathbf{Ax} - \mathbf{b} \geq \mathbf{0}$$

- ▶ solutions of **least-squares problems**:

$$\min \|\mathbf{Ax} - \mathbf{b}\|_2 \Leftrightarrow \min \|\mathbf{r}\|_2 \wedge \mathbf{r} := \mathbf{b} - \mathbf{Ax}$$

Another simple example in machine learning (ML) is known as **support vector machine** (SVM).

SVM: comparing apples and oranges pears



SVM: comparing apples and oranges pears

Example of a SVM for the task to distinguish **apples** und **pears**: use **features**!

apple		pear	
length	thickness	length	thickness
5.5	6.0	9.0	6.4
5.0	5.0	11.0	7.0
5.0	6.0	10.6	5.4
6.5	7.0	9.0	4.6
7.0	7.0	6.0	4.4
5.1	4.9	8.0	6.0
5.5	4.8	9.0	6.0
7.6	8.0	7.0	4.4
6.0	6.0	9.0	8.2
6.5	7.0	10.0	5.8
8.0	8.7	8.0	6.2

SVM: comparing apples and oranges pears

Example of a SVM for the task to distinguish **apples** und **pears**: use **features**!

apple		pear	
length	thickness	length	thickness
5.5	6.0	9.0	6.4
5.0	5.0	11.0	7.0
5.0	6.0	10.6	5.4
6.5	7.0	9.0	4.6
7.0	7.0	6.0	4.4
5.1	4.9	8.0	6.0
5.5	4.8	9.0	6.0
7.6	8.0	7.0	4.4
6.0	6.0	9.0	8.2
6.5	7.0	10.0	5.8
8.0	8.7	8.0	6.2

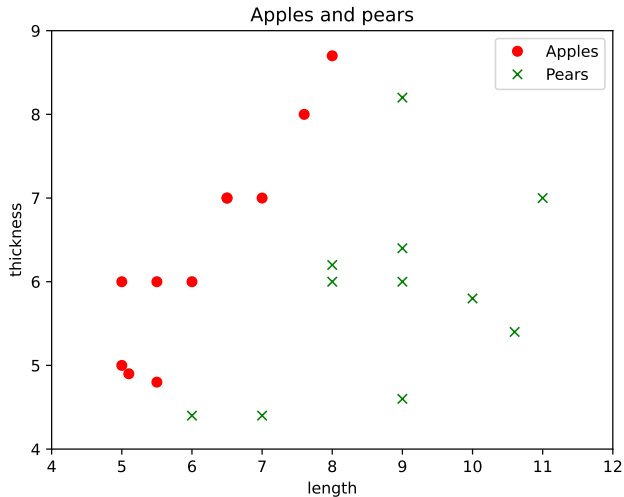
SVM: comparing apples and oranges pears

Example of a SVM for the task to distinguish **apples** und **pears**: use **features**!

apple		pear	
length	thickness	length	thickness
5.5	6.0	9.0	6.4
5.0	5.0	11.0	7.0
5.0	6.0	10.6	5.4
6.5	7.0	9.0	4.6
7.0	7.0	6.0	4.4
5.1	4.9	8.0	6.0
5.5	4.8	9.0	6.0
7.6	8.0	7.0	4.4
6.0	6.0	9.0	8.2
6.5	7.0	10.0	5.8
8.0	8.7	8.0	6.2

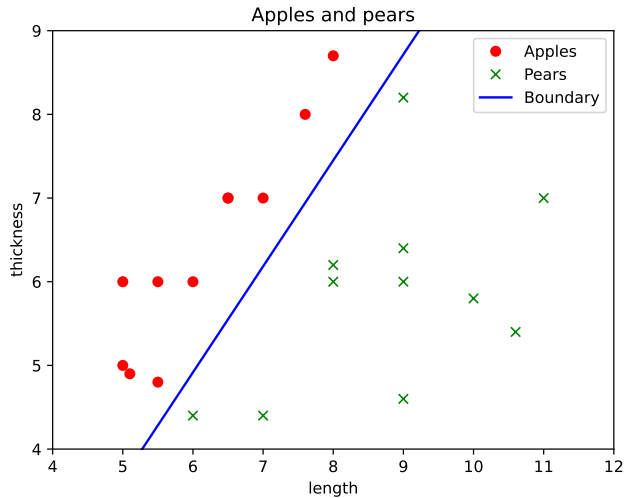
Fruits as vectors in $\mathbb{R}^2$

Pears rather long than round $\leadsto$ simple **classification** possible.



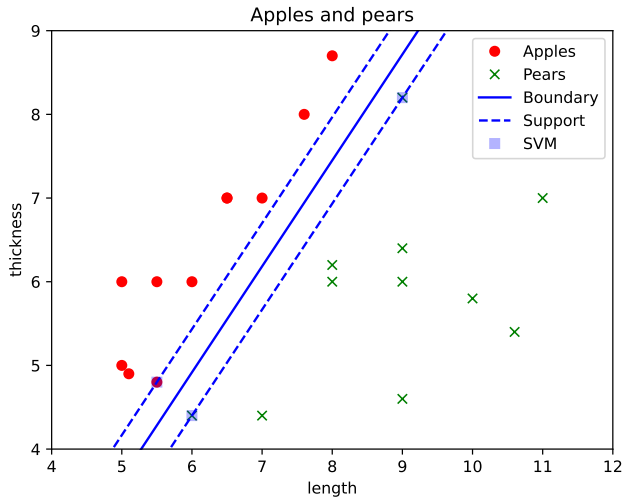
Fruits as vectors in $\mathbb{R}^2$

Pears rather long than round $\leadsto$ simple **classification** possible.



Fruits as vectors in $\mathbb{R}^2$

Pears rather long than round $\rightsquigarrow$ simple **classification** possible.



Separating hyperplane

Fruit corresponds to **vector** in $\mathbb{R}^2$, marked vectors “**support**” hyperplane $\rightsquigarrow$ **support vectors**. Seek separating hyperplane in the form (note the sign change in b)

$$\ell(\mathbf{x}) := \mathbf{w}^T \mathbf{x} + b = \mathbf{x}^T \mathbf{w} + b.$$

Separating hyperplane

Fruit corresponds to **vector** in $\mathbb{R}^2$, marked vectors “**support**” hyperplane $\rightsquigarrow$ **support vectors**. Seek separating hyperplane in the form (note the sign change in b)

$$\ell(\mathbf{x}) := \mathbf{w}^T \mathbf{x} + b = \mathbf{x}^T \mathbf{w} + b.$$

Known support vectors: separating line **computable**. Suppose that apples return +1, pears -1:

$$\begin{pmatrix} 5.5 & 4.8 \\ 6.0 & 4.4 \\ 9.0 & 8.2 \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} b = \begin{pmatrix} 1 \\ -1 \\ -1 \end{pmatrix},$$

Separating hyperplane

Fruit corresponds to **vector** in $\mathbb{R}^2$, marked vectors “**support**” hyperplane $\rightsquigarrow$ **support vectors**. Seek separating hyperplane in the form (note the sign change in b)

$$\ell(\mathbf{x}) := \mathbf{w}^T \mathbf{x} + b = \mathbf{x}^T \mathbf{w} + b.$$

Known support vectors: separating line **computable**. Suppose that apples return +1, pears -1:

$$\begin{pmatrix} 5.5 & 4.8 \\ 6.0 & 4.4 \\ 9.0 & 8.2 \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} b = \begin{pmatrix} 1 \\ -1 \\ -1 \end{pmatrix},$$

Reorder $\rightsquigarrow$ **linear system** for the three unknowns,

$$\begin{pmatrix} 5.5 & 4.8 & 1 \\ 6.0 & 4.4 & 1 \\ 9.0 & 8.2 & 1 \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \\ b \end{pmatrix} = \begin{pmatrix} 1 \\ -1 \\ -1 \end{pmatrix} \Rightarrow \begin{pmatrix} w_1 \\ w_2 \\ b \end{pmatrix} = \begin{pmatrix} -76/31 \\ 60/31 \\ 161/31 \end{pmatrix} \approx \begin{pmatrix} -2.4516129032258065 \\ 1.9354838709677420 \\ 5.1935483870967740 \end{pmatrix}.$$

Classification, general computation

Size of the margin of the area defined by the lines:

$$\frac{\langle \mathbf{x}_1 - \mathbf{x}_2, \mathbf{w} \rangle}{\|\mathbf{w}\|_2} = \frac{\langle \mathbf{x}_1, \mathbf{w} \rangle}{\|\mathbf{w}\|_2} - \frac{\langle \mathbf{x}_2, \mathbf{w} \rangle}{\|\mathbf{w}\|_2} = \frac{+1 - b}{\|\mathbf{w}\|_2} - \frac{-1 - b}{\|\mathbf{w}\|_2} = \frac{2}{\|\mathbf{w}\|_2} \approx 0.6402990981983214.$$

Classification, general computation

Size of the margin of the area defined by the lines:

$$\frac{\langle \mathbf{x}_1 - \mathbf{x}_2, \mathbf{w} \rangle}{\|\mathbf{w}\|_2} = \frac{\langle \mathbf{x}_1, \mathbf{w} \rangle}{\|\mathbf{w}\|_2} - \frac{\langle \mathbf{x}_2, \mathbf{w} \rangle}{\|\mathbf{w}\|_2} = \frac{+1 - b}{\|\mathbf{w}\|_2} - \frac{-1 - b}{\|\mathbf{w}\|_2} = \frac{2}{\|\mathbf{w}\|_2} \approx 0.6402990981983214.$$

Other points have a signed distance larger/smaller than ± 1 . Define y_i as ± 1 , then

$$y_i \ell(\mathbf{x}_i) = y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \quad \forall i = 1, \dots, N.$$

Classification, general computation

Size of the margin of the area defined by the lines:

$$\frac{\langle \mathbf{x}_1 - \mathbf{x}_2, \mathbf{w} \rangle}{\|\mathbf{w}\|_2} = \frac{\langle \mathbf{x}_1, \mathbf{w} \rangle}{\|\mathbf{w}\|_2} - \frac{\langle \mathbf{x}_2, \mathbf{w} \rangle}{\|\mathbf{w}\|_2} = \frac{+1 - b}{\|\mathbf{w}\|_2} - \frac{-1 - b}{\|\mathbf{w}\|_2} = \frac{2}{\|\mathbf{w}\|_2} \approx 0.6402990981983214.$$

Other points have a signed distance larger/smaller than ± 1 . Define y_i as ± 1 , then

$$y_i \ell(\mathbf{x}_i) = y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \quad \forall i = 1, \dots, N.$$

Weights $\mathbf{w}$ and bias b uniquely determined as solution of the optimization problem

$$\min_{\mathbf{w}, b} \{ \|\mathbf{w}\|_2 \mid y_i \ell(\mathbf{x}_i) \geq 1 \quad \forall i = 1, \dots, N \}.$$

Classification, general computation

Size of the margin of the area defined by the lines:

$$\frac{\langle \mathbf{x}_1 - \mathbf{x}_2, \mathbf{w} \rangle}{\|\mathbf{w}\|_2} = \frac{\langle \mathbf{x}_1, \mathbf{w} \rangle}{\|\mathbf{w}\|_2} - \frac{\langle \mathbf{x}_2, \mathbf{w} \rangle}{\|\mathbf{w}\|_2} = \frac{+1 - b}{\|\mathbf{w}\|_2} - \frac{-1 - b}{\|\mathbf{w}\|_2} = \frac{2}{\|\mathbf{w}\|_2} \approx 0.6402990981983214.$$

Other points have a signed distance larger/smaller than ± 1 . Define y_i as ± 1 , then

$$y_i \ell(\mathbf{x}_i) = y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \quad \forall i = 1, \dots, N.$$

Weights $\mathbf{w}$ and bias b uniquely determined as solution of the optimization problem

$$\min_{\mathbf{w}, b} \{ \|\mathbf{w}\|_2 \mid y_i \ell(\mathbf{x}_i) \geq 1 \quad \forall i = 1, \dots, N \}.$$

Dual formulation, non-linear form of separation: scalar product $\mathbf{x}^T \mathbf{x}' \rightsquigarrow$ kernel $K(\mathbf{x}, \mathbf{x}')$

- ▶ Gaussian kernel $K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|_2^2)$
- ▶ hyperbolic tangent $K(\mathbf{x}, \mathbf{x}') = \tanh(\gamma \mathbf{x}^T \mathbf{x}' - r)$ for certain $\gamma, r > 0$

SVM software example: Fisher's Iris data set



Data set:

- ▶ Iris setosa
- ▶ Iris versicolor
- ▶ Iris virginica

Properties:

- ▶ 150 samples
- ▶ 50 for each species
- ▶ Sepal length in cm
- ▶ Sepal width in cm
- ▶ Petal length in cm
- ▶ Petal width in cm

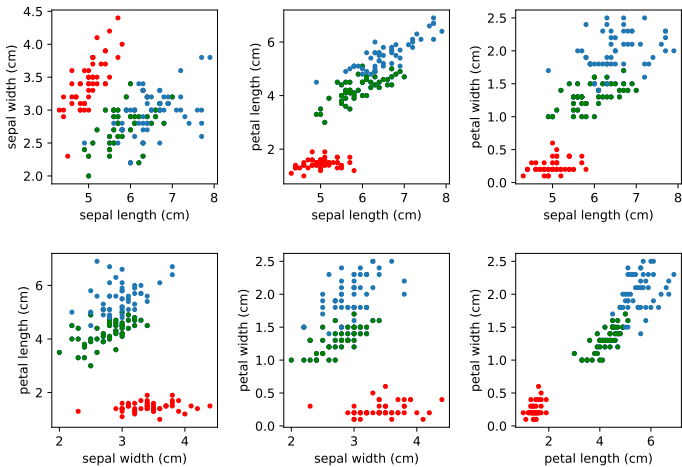
Iris setosa, CC BY-SA 3.0, <https://commons.wikimedia.org/w/index.php?curid=170298>,

Iris virginica, by Frank Mayfield, CC BY-SA 2.0, <https://commons.wikimedia.org/w/index.php?curid=9805580>,

Iris versicolor, CC BY-SA 3.0, <https://commons.wikimedia.org/w/index.php?curid=248095>.

SVM software example: Fisher's Iris data set

Iris setosa (red), Iris versicolor (blue), Iris virginica (green)

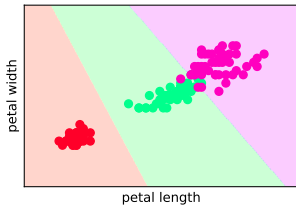


SVM software example: Fisher's Iris data set

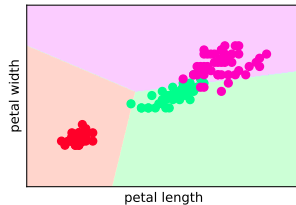
```
1 import numpy as np
2 from sklearn import svm, datasets
3 # import data, use only last two features
4 iris = datasets.load_iris()
5 X = iris.data[:, 2:]
6 y = iris.target # values 0, 1, 2
7 # different SVC
8 svc = svm.SVC(kernel='linear', gamma=1.0, C=1.0).fit(X, y)
9 rbf_svc = svm.SVC(kernel='rbf', gamma=1.0, C=1.0).fit(X, y)
10 poly_svc = svm.SVC(kernel='poly', gamma='auto',
11                    degree=3, coef0=1.0, C=1.0).fit(X, y)
12 lin_svc = svm.LinearSVC(max_iter=5000, C=1.0, dual=True).fit(X, y)
13 # use simple data point and classify it
14 x_test = np.ones((1, 2))
15 y_svc = svc.predict(x_test) # = 0, corresponds to Iris setosa
16 y_rbf_svc = rbf_svc.predict(x_test) # = 0
17 y_poly_svc = poly_svc.predict(x_test) # = 0
18 y_lin_svc = lin_svc.predict(x_test) # = 0
```

SVM software example: Fisher's Iris data set

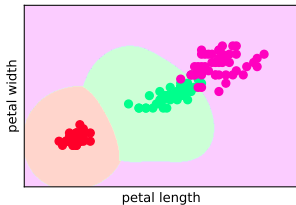
SVC w/ linear kernel



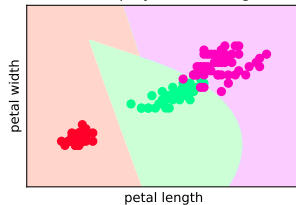
LinearSVC (linear kernel)



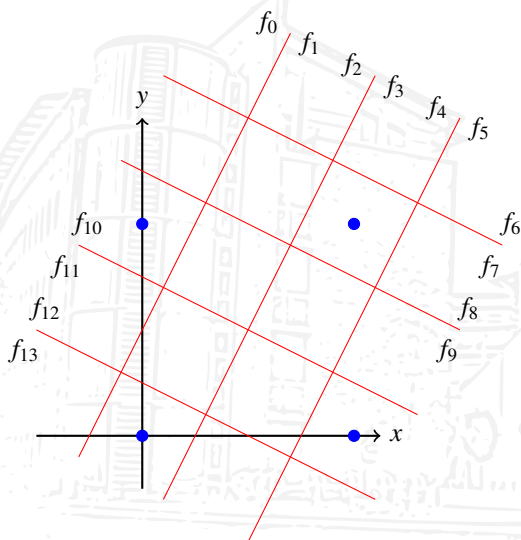
SVC w/ RBF kernel



SVC w/ poly. kernel, deg. 3



Separable Boolean functions



f_0 :	$\neg x \wedge y$,	inhibition of y ,
f_1 :	$y \rightarrow x$,	implication,
f_2 :	$\neg x$,	negation of x ,
f_3 :	x ,	projection onto x ,
f_4 :	$x \rightarrow y$,	implication,
f_5 :	$x \wedge \neg y$,	inhibition of x ,
f_6 :	0,	constant FALSE,
f_7 :	1,	constant TRUE,
f_8 :	$x \wedge y$,	AND,
f_9 :	$\neg x \vee \neg y$,	NAND,
f_{10} :	y ,	projection onto y ,
f_{11} :	$\neg y$,	negation of y ,
f_{12} :	$x \vee y$,	OR,
f_{13} :	$\neg x \wedge \neg y$,	NOR.

Separable Boolean functions

n	number of Boolean functions	number of threshold functions
0	2	2
1	4	4
2	16	14
3	256	104
4	65536	1882
5	4294967296	94572
6	18446744073709551616	15028134
7	$\approx 3.402823669209385 \cdot 10^{38}$	8378070864
8	$\approx 1.157920892373162 \cdot 10^{77}$	17561539552946
9	$\approx 1.3407807929942597 \cdot 10^{154}$	144130531453121108

Heaviside function

Apples separated by straight line from pears. Obtain function that marks apples: combine w/ **Heaviside function** (step function) H ,

$$H(\mathbf{w}^T \mathbf{x} + b), \quad H(x) := \begin{cases} 0, & x < 0, \\ 1, & x \geq 0. \end{cases}$$

Heaviside function

Apples separated by straight line from pears. Obtain function that marks apples: combine w/ **Heaviside function** (step function) H ,

$$H(\mathbf{w}^T \mathbf{x} + b), \quad H(x) := \begin{cases} 0, & x < 0, \\ 1, & x \geq 0. \end{cases}$$

Results in **characteristic function of apples** (1 if apple, 0 else).

Heaviside function

Apples separated by straight line from pears. Obtain function that marks apples: combine w/ **Heaviside function** (step function) H ,

$$H(\mathbf{w}^T \mathbf{x} + b), \quad H(x) := \begin{cases} 0, & x < 0, \\ 1, & x \geq 0. \end{cases}$$

Results in **characteristic function of apples** (1 if apple, 0 else).

Apart from zero **differentiable w/ derivative zero** and in zero **not continuous/differentiable**.

Heaviside function

Apples separated by straight line from pears. Obtain function that marks apples: combine w/ **Heaviside function** (step function) H ,

$$H(\mathbf{w}^T \mathbf{x} + b), \quad H(x) := \begin{cases} 0, & x < 0, \\ 1, & x \geq 0. \end{cases}$$

Results in **characteristic function of apples** (1 if apple, 0 else).

Apart from zero **differentiable w/ derivative zero** and in zero **not continuous/differentiable**.

Mostly **modified** $\rightsquigarrow$ line treated as “unknown” (classification into three parts):

$$M(x) := \begin{cases} 0, & x < 0, \\ \frac{1}{2}, & x = 0, \\ 1, & x > 0. \end{cases}$$

Logistic function

Better: information on closeness to separating line, preferably range bounded, e.g., in $[0, 1]$.

Classical variant: **logistic function** (sigmoid function)

$$\text{Logistic}(x) := (1 + e^{-x})^{-1}, \quad \text{Logistic} : \mathbb{R} \rightarrow (0, 1).$$

Logistic function

Better: information on closeness to separating line, preferably range bounded, e.g., in $[0, 1]$.

Classical variant: **logistic function** (sigmoid function)

$$\text{Logistic}(x) := (1 + e^{-x})^{-1}, \quad \text{Logistic} : \mathbb{R} \rightarrow (0, 1).$$

The more $\text{Logistic}(x) \approx 1$, the more apple-like.

Logistic function

Better: information on closeness to separating line, preferably range bounded, e.g., in $[0, 1]$.

Classical variant: **logistic function** (sigmoid function)

$$\text{Logistic}(x) := (1 + e^{-x})^{-1}, \quad \text{Logistic} : \mathbb{R} \rightarrow (0, 1).$$

The more $\text{Logistic}(x) \approx 1$, the more apple-like.

Analytical, **Logistic'** = **Logistic** · (1 – **Logistic**).

Logistic function

Better: information on closeness to separating line, preferably range bounded, e.g., in $[0, 1]$.
Classical variant: **logistic function** (sigmoid function)

$$\text{Logistic}(x) := (1 + e^{-x})^{-1}, \quad \text{Logistic} : \mathbb{R} \rightarrow (0, 1).$$

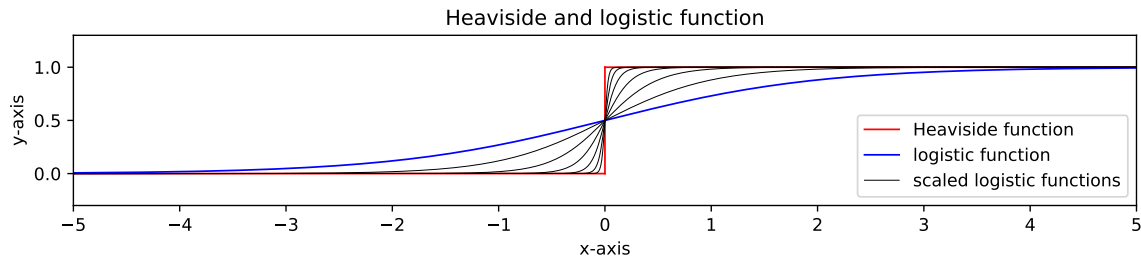
The more $\text{Logistic}(x) \approx 1$, the more apple-like.

Analytical, $\text{Logistic}' = \text{Logistic} \cdot (1 - \text{Logistic})$.

Shape parameter changes **slope** at zero, in the pointwise limit we obtain modified Heaviside:

$$\lim_{k \rightarrow \infty} \text{Logistic}(kx) = \lim_{k \rightarrow \infty} (1 + e^{-kx})^{-1} = M(x).$$

Heaviside and logistic function



In black: scaled logistic functions

$$\text{Logistic}_k(x) := \text{Logistic}(kx) = \frac{1}{1 + e^{-kx}}$$

for $k \in 2^{\{1,2,3,4,5,6\}}$.

Sign and hyperbolic tangent

Also **classify pears** $\rightsquigarrow$ **sign function** $\text{Sign} := 2M - 1$,

$$\text{Sign}(x) := (2M - 1)(x) = \begin{cases} -1, & x < 0, \\ 0, & x = 0, \\ 1, & x > 0. \end{cases}$$

Apples have value $+1$, pears have value -1 , border has value 0 . Derivative a.e. zero.

Sign and hyperbolic tangent

Also **classify pears** $\rightsquigarrow$ **sign function** $\text{Sign} := 2M - 1$,

$$\text{Sign}(x) := (2M - 1)(x) = \begin{cases} -1, & x < 0, \\ 0, & x = 0, \\ 1, & x > 0. \end{cases}$$

Apples have value +1, pears have value -1, border has value 0. Derivative a.e. zero.

Analytical variant:

$$2\text{Logistic}(2x) - 1 = \frac{2}{1 + e^{-2x}} - 1 = \frac{2}{1 + e^{-2x}} - \frac{1 + e^{-2x}}{1 + e^{-2x}} = \frac{1 - e^{-2x}}{1 + e^{-2x}}$$

Sign and hyperbolic tangent

Also **classify pears** $\rightsquigarrow$ **sign function** $\text{Sign} := 2M - 1$,

$$\text{Sign}(x) := (2M - 1)(x) = \begin{cases} -1, & x < 0, \\ 0, & x = 0, \\ 1, & x > 0. \end{cases}$$

Apples have value $+1$, pears have value -1 , border has value 0 . Derivative a.e. zero.

Analytical variant:

$$2\text{Logistic}(2x) - 1 = \frac{2}{1 + e^{-2x}} - 1 = \frac{2}{1 + e^{-2x}} - \frac{1 + e^{-2x}}{1 + e^{-2x}} = \frac{1 - e^{-2x}}{1 + e^{-2x}} = \frac{1 - e^{-2x}}{1 + e^{-2x}} \cdot \frac{e^x}{e^x} = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Sign and hyperbolic tangent

Also **classify pears** $\rightsquigarrow$ **sign function** $\text{Sign} := 2M - 1$,

$$\text{Sign}(x) := (2M - 1)(x) = \begin{cases} -1, & x < 0, \\ 0, & x = 0, \\ 1, & x > 0. \end{cases}$$

Apples have value $+1$, pears have value -1 , border has value 0 . Derivative a.e. zero.

Analytical variant:

$$2\text{Logistic}(2x) - 1 = \frac{2}{1 + e^{-2x}} - 1 = \frac{2}{1 + e^{-2x}} - \frac{1 + e^{-2x}}{1 + e^{-2x}} = \frac{1 - e^{-2x}}{1 + e^{-2x}} = \frac{1 - e^{-2x}}{1 + e^{-2x}} \cdot \frac{e^x}{e^x} = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \text{TanH}(x),$$

hyperbolic tangent $\text{TanH} : \mathbb{R} \rightarrow (-1, 1)$.

Sign and hyperbolic tangent

Also **classify pears** $\rightsquigarrow$ **sign function** $\text{Sign} := 2M - 1$,

$$\text{Sign}(x) := (2M - 1)(x) = \begin{cases} -1, & x < 0, \\ 0, & x = 0, \\ 1, & x > 0. \end{cases}$$

Apples have value $+1$, pears have value -1 , border has value 0 . Derivative a.e. zero.

Analytical variant:

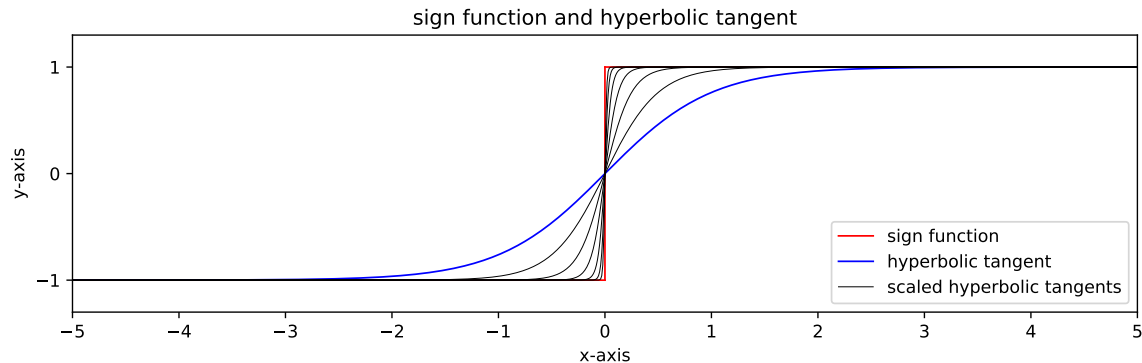
$$2\text{Logistic}(2x) - 1 = \frac{2}{1 + e^{-2x}} - 1 = \frac{2}{1 + e^{-2x}} - \frac{1 + e^{-2x}}{1 + e^{-2x}} = \frac{1 - e^{-2x}}{1 + e^{-2x}} \cdot \frac{e^x}{e^x} = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \text{TanH}(x),$$

hyperbolic tangent $\text{TanH} : \mathbb{R} \rightarrow (-1, 1)$.

Shape parameter $k > 0$, $\text{TanH}_k(x) := \text{TanH}(kx)$, pointwise limit $k \rightarrow \infty$ gives Sign .

Derivative: $\text{TanH}' = 1 - \text{TanH}^2$.

Sign and hyperbolic tangent



In black: scaled hyperbolic tangents for $k \in 2^{\{1,2,3,4,5,6\}}$,

$$\text{TanH}_k(x) = \text{TanH}(kx) = \frac{e^{kx} - e^{-kx}}{e^{kx} + e^{-kx}}.$$

ReLU and related activation functions

Nowadays: fast computation of activation function/derivative,

$$\text{ReLU}(x) := (x)_+ := \max(x, 0) = \begin{cases} 0, & x \leq 0, \\ x, & x > 0. \end{cases}$$

ReLU abbreviates “**Rectified Linear Unit**”.

ReLU and related activation functions

Nowadays: fast computation of activation function/derivative,

$$\text{ReLU}(x) := (x)_+ := \max(x, 0) = \begin{cases} 0, & x \leq 0, \\ x, & x > 0. \end{cases}$$

ReLU abbreviates “**Rectified Linear Unit**”.

$\text{ReLU} : \mathbb{R} \rightarrow \mathbb{R}_0^+$, **differentiable outside zero** $\rightsquigarrow$ derivative: (modified) Heaviside function.

ReLU and related activation functions

Nowadays: fast computation of activation function/derivative,

$$\text{ReLU}_\alpha(x) := \text{leaky ReLU}(x) := \max(x, 0) + \min(0, \alpha x) = \begin{cases} \alpha x, & x \leq 0, \\ x, & x > 0. \end{cases}$$

ReLU abbreviates “**Rectified Linear Unit**”.

$\text{ReLU} : \mathbb{R} \rightarrow \mathbb{R}_0^+$, **differentiable outside zero** $\rightsquigarrow$ derivative: (modified) Heaviside function.

Variants of ReLU (mostly antiderivatives of Heaviside variants):

- ▶ **leaky ReLU** for $\alpha > 0$, mostly $\alpha = 0.01$

ReLU and related activation functions

Nowadays: fast computation of activation function/derivative,

$$\text{ELU}_{\alpha}(x) := \begin{cases} \alpha(e^x - 1), & x \leq 0, \\ x, & x > 0. \end{cases}$$

ReLU abbreviates “**Rectified Linear Unit**”.

$\text{ReLU} : \mathbb{R} \rightarrow \mathbb{R}_0^+$, **differentiable outside zero** $\rightsquigarrow$ derivative: (modified) Heaviside function.

Variants of ReLU (mostly antiderivatives of Heaviside variants):

- ▶ **leaky ReLU** for $\alpha > 0$, mostly $\alpha = 0.01$
- ▶ **Exponential Linear Unit** ($\text{ELU}(x) > -\alpha$, C^1 for $\alpha = 1$)

ReLU and related activation functions

Nowadays: fast computation of activation function/derivative,

$$\text{SoftPlus}_k(x) := \frac{\ln(1 + e^{kx})}{k}.$$

ReLU abbreviates “**Rectified Linear Unit**”.

ReLU : $\mathbb{R} \rightarrow \mathbb{R}_0^+$, **differentiable outside zero** $\rightsquigarrow$ derivative: (modified) Heaviside function.

Variants of ReLU (mostly antiderivatives of Heaviside variants):

- ▶ **leaky ReLU** for $\alpha > 0$, mostly $\alpha = 0.01$
- ▶ **Exponential Linear Unit** ($\text{ELU}(x) > -\alpha$, C^1 for $\alpha = 1$)
- ▶ **SoftPlus** (analytical, derivative is logistic function, often shifted by $-\ln(2)/k$)

ReLU and related activation functions

Nowadays: fast computation of activation function/derivative,

$$\text{Swish}_k(x) := x \cdot \text{Logistic}_k(x).$$

ReLU abbreviates “**Rectified Linear Unit**”.

$\text{ReLU} : \mathbb{R} \rightarrow \mathbb{R}_0^+$, **differentiable outside zero** $\rightsquigarrow$ derivative: (modified) Heaviside function.

Variants of ReLU (mostly antiderivatives of Heaviside variants):

- ▶ **leaky ReLU** for $\alpha > 0$, mostly $\alpha = 0.01$
- ▶ **Exponential Linear Unit** ($\text{ELU}(x) > -\alpha$, C^1 for $\alpha = 1$)
- ▶ **SoftPlus** (analytical, derivative is logistic function, often shifted by $-\ln(2)/k$)
- ▶ **Swish** ($\text{ReLU}(x) = x \cdot H(x) \rightsquigarrow \text{Swish}_k(x) := x \cdot \text{Logistic}_k(x)$)

ReLU and related activation functions

Nowadays: fast computation of activation function/derivative,

$$\text{ReLU}(x) := (x)_+ := \max(x, 0) = \begin{cases} 0, & x \leq 0, \\ x, & x > 0. \end{cases}$$

ReLU abbreviates “**Rectified Linear Unit**”.

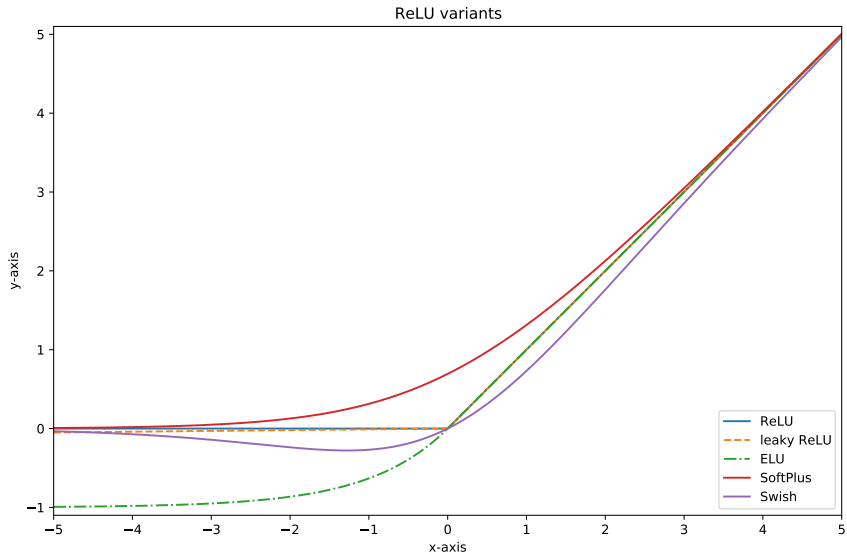
$\text{ReLU} : \mathbb{R} \rightarrow \mathbb{R}_0^+$, **differentiable outside zero** $\rightsquigarrow$ derivative: (modified) Heaviside function.

Variants of ReLU (mostly antiderivatives of Heaviside variants):

- ▶ **leaky ReLU** for $\alpha > 0$, mostly $\alpha = 0.01$
- ▶ **Exponential Linear Unit** ($\text{ELU}(x) > -\alpha$, C^1 for $\alpha = 1$)
- ▶ **SoftPlus** (analytical, derivative is logistic function, often shifted by $-\ln(2)/k$)
- ▶ **Swish** ($\text{ReLU}(x) = x \cdot H(x) \rightsquigarrow \text{Swish}_k(x) := x \cdot \text{Logistic}_k(x)$)

Next picture: all five variants for their natural parameters, where $k = 1$.

ReLU and related activation functions



Abs family

Seldom used (but my favorite): $\text{Abs}(x) := |x|$, antiderivative of Sign and $\text{Abs}(x) = x \cdot \text{Sign}(x)$.



Abs family

Seldom used (but my favorite): $\text{Abs}(x) := |x|$, antiderivative of Sign and $\text{Abs}(x) = x \cdot \text{Sign}(x)$.

New variants by replicating approaches for ReLU:

Abs family

Seldom used (but my favorite): $\text{Abs}(x) := |x|$, antiderivative of Sign and $\text{Abs}(x) = x \cdot \text{Sign}(x)$.

New variants by replicating approaches for ReLU:

- ▶ SoftAbs from SoftSign (latter belongs to Sign family):

$$\text{SoftAbs}_k(x) := x \cdot \text{SoftSign}_k, \quad \text{where} \quad \text{SoftSign}_k(x) := \frac{kx}{1 + |kx|}$$

Abs family

Seldom used (but my favorite): **Abs**(x) $:= |x|$, antiderivative of Sign and $\text{Abs}(x) = x \cdot \text{Sign}(x)$.

New variants by replicating approaches for ReLU:

- ▶ SoftAbs from SoftSign (latter belongs to Sign family):

$$\text{SoftAbs}_k(x) := x \cdot \text{SoftSign}_k, \quad \text{where} \quad \text{SoftSign}_k(x) := \frac{kx}{1 + |kx|}$$

- ▶ LOCo, antiderivative of TanH:

$$\text{LOCo}_k(x) := \frac{\ln(\cosh(kx))}{k}, \quad \text{LOCo}'_k = \text{TanH}_k$$

Abs family

Seldom used (but my favorite): $\text{Abs}(x) := |x|$, antiderivative of Sign and $\text{Abs}(x) = x \cdot \text{Sign}(x)$.

New variants by replicating approaches for ReLU:

- ▶ SoftAbs from SoftSign (latter belongs to Sign family):

$$\text{SoftAbs}_k(x) := x \cdot \text{SoftSign}_k, \quad \text{where} \quad \text{SoftSign}_k(x) := \frac{kx}{1 + |kx|}$$

- ▶ LOCo, antiderivative of TanH:

$$\text{LOCo}_k(x) := \frac{\ln(\cosh(kx))}{k}, \quad \text{LOCo}'_k = \text{TanH}_k$$

- ▶ $\text{Twist}_k(x) := x \cdot \text{TanH}_k(x)$

Abs family

Seldom used (but my favorite): $\text{Abs}(x) := |x|$, antiderivative of Sign and $\text{Abs}(x) = x \cdot \text{Sign}(x)$.

New variants by replicating approaches for ReLU:

- ▶ SoftAbs from SoftSign (latter belongs to Sign family):

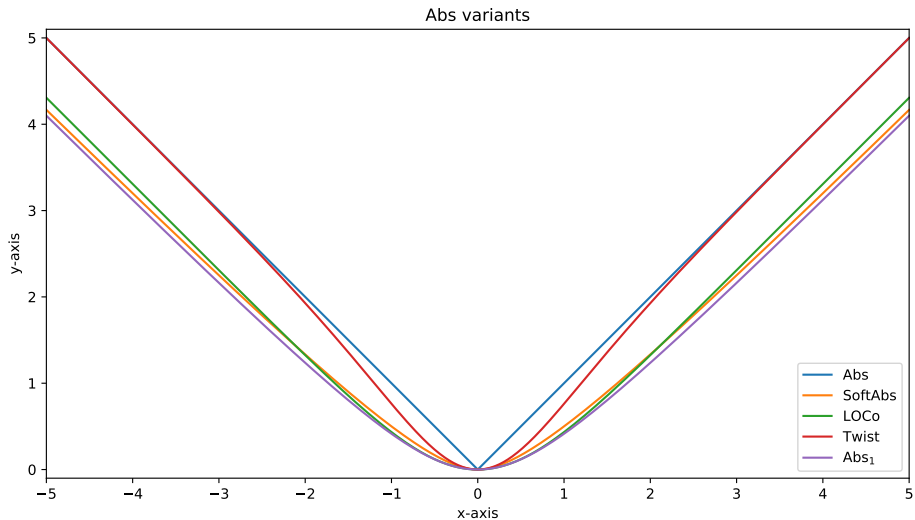
$$\text{SoftAbs}_k(x) := x \cdot \text{SoftSign}_k, \quad \text{where} \quad \text{SoftSign}_k(x) := \frac{kx}{1 + |kx|}$$

- ▶ LOCo, antiderivative of TanH:

$$\text{LOCo}_k(x) := \frac{\ln(\cosh(kx))}{k}, \quad \text{LOCo}'_k = \text{TanH}_k$$

- ▶ $\text{Twist}_k(x) := x \cdot \text{TanH}_k(x)$
- ▶ $\text{Abs}(x) = \sqrt{x^2} \rightsquigarrow \text{Abs}_\alpha(x) := \sqrt{x^2 + \alpha^2} - \alpha$, where $\alpha > 0$

Abs family



Grouping activation functions—the known ones

function	derivative	properties
H/M	zero a.e.	monotone, range $\{0, 1/2, 1\}$, discontinuous
Logistic_k	$k \text{Logistic}_k(1 - \text{Logistic}_k)$	monotone, range $(0, 1)$, C^ω
Sign	zero a.e.	monotone, range $\{-1, 0, 1\}$, discontinuous
SoftSign_k	$k(1 + kx)^{-2}$	monotone, range $(-1, 1)$, C^1
TanH_k	$k(1 - \text{TanH}_k^2)$	monotone, range $(-1, 1)$, C^ω
ReLU	H/M a.e.	monotone, range $[0, \infty)$, C
ReLU_α	$H(x) + \alpha H(-x)$ a.e.	monotone, range $(-\infty, \infty)$, C
ELU_α	$H(x) + \alpha \exp(x)H(-x)$ a.e.	monotone, range $(-\alpha, \infty)$, C (C^1 if $\alpha = 1$)
SoftPlus_k	Logistic_k	monotone, range $(0, \infty)$, C^ω
Swish_k	in terms of Logistic_k	monotone, range $[-c/k, \infty)$, $c \approx 0.278465$, C^ω

$C = C^0$; C^k : k times continuously differentiable, $k \in \mathbb{N}_0 \cup \{\infty\}$; C^ω : analytical

Grouping activation functions—the new ones

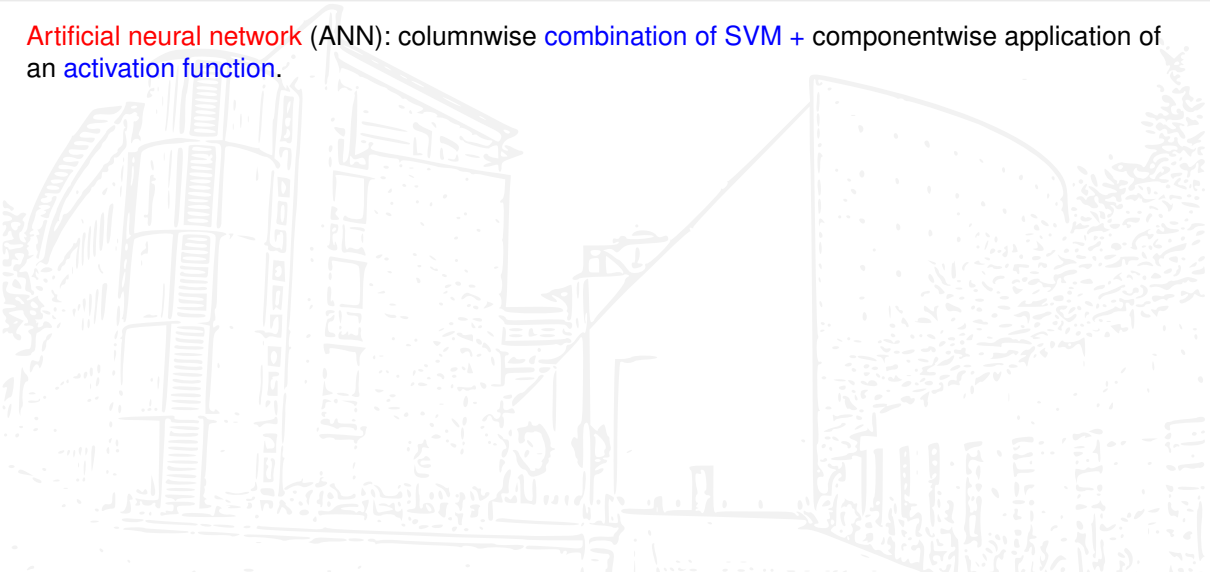
Abs family: all activation functions even, i.e., $f(x) = f(-x)$; range $[0, \infty)$; zero at zero.

function	derivative	properties
Abs	Sign a.e.	positive monotone, C
SoftAbs _k	exercise	exercise
LOCo _k	TanH _k	positive monotone, C^ω , asymptotes: $ x - \ln(2)/k$
Twist _k	in terms of TanH _k	positive monotone, C^ω , asymptotes: $ x $
Abs _α	$x/\sqrt{x^2 + \alpha^2}$	positive monotone, C^ω , asymptotes: $ x - \alpha$

$C = C^0$; C^k : k times continuously differentiable, $k \in \mathbb{N}_0 \cup \{\infty\}$; C^ω : analytical

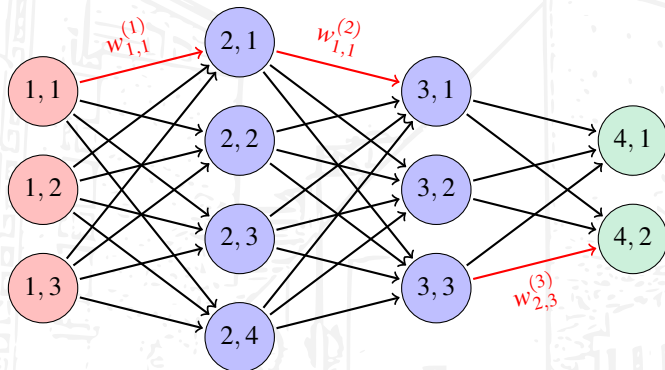
Artificial neural network (ANN)

Artificial neural network (ANN): columnwise combination of SVM + componentwise application of an activation function.



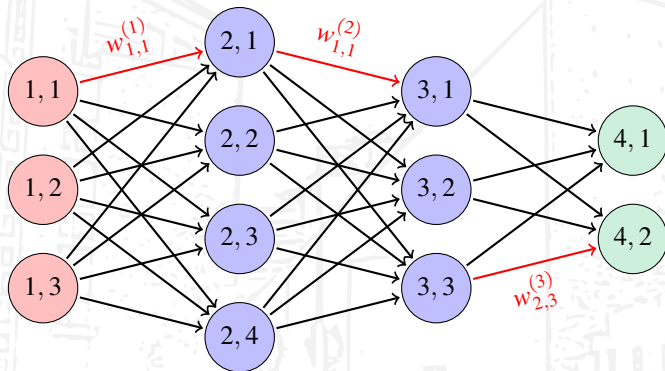
Artificial neural network (ANN)

Artificial neural network (ANN): columnwise combination of SVM + componentwise application of an activation function.



Artificial neural network (ANN)

Artificial neural network (ANN): columnwise combination of SVM + componentwise application of an activation function.



Function $\mathbf{f} : \mathbb{R}^3 \rightarrow \mathbb{R}^2$; input $\mathbf{x} \in \mathbb{R}^3 \rightsquigarrow$ red circles; output $\mathbf{f}(\mathbf{x}) \in \mathbb{R}^2 \rightsquigarrow$ green circles.

Terminology

- ▶ red layer: “input layer”
- ▶ green layer: “output layer”
- ▶ all other layers: “hidden layers”
- ▶ colored circles: “neurons”

Terminology

- ▶ red layer: “input layer”
- ▶ green layer: “output layer”
- ▶ all other layers: “hidden layers”
- ▶ colored circles: “neurons”

Evaluation of the function $\mathbf{f} \rightsquigarrow$ **loop of similar computations:**

$$\mathbf{a}_1 := \mathbf{x}, \quad \mathbf{z}_i := \mathbf{W}_i \mathbf{a}_i + \mathbf{b}_i, \quad \mathbf{a}_{i+1} := a_i(\mathbf{z}_i), \quad i = 1, \dots, 3, \quad \mathbf{f}(\mathbf{x}) := \mathbf{a}_4.$$

Terminology

- ▶ red layer: “input layer”
- ▶ green layer: “output layer”
- ▶ all other layers: “hidden layers”
- ▶ colored circles: “neurons”

Evaluation of the function $\mathbf{f} \rightsquigarrow$ **loop of similar computations:**

$$\mathbf{a}_1 := \mathbf{x}, \quad \mathbf{z}_i := \mathbf{W}_i \mathbf{a}_i + \mathbf{b}_i, \quad \mathbf{a}_{i+1} := a_i(\mathbf{z}_i), \quad i = 1, \dots, 3, \quad \mathbf{f}(\mathbf{x}) := \mathbf{a}_4.$$

- ▶ edges: entries of **weight matrices** $\mathbf{W}_i$, $i = 1, \dots, 3$
- ▶ weights of edges leading to a neuron in **row**
- ▶ per non-input layer **bias vector** $\mathbf{b}_i$, $i = 1, \dots, 3$
- ▶ function a_i (componentenwise) **activation function** from last section
- ▶ **activation** $\mathbf{a}_{i+1} \in \mathbb{R}^{n_{i+1}}$ same dimensionality as input $\mathbf{z}_i$

General ANN $\rightsquigarrow$ Feedforward-Network (FNN)

General ANN uniquely defined by **structure**:

- ▶ number m of layers
- ▶ number of neurons per layer, uniquely described by $\mathbf{n} \in \mathbb{N}^m$

General ANN $\rightsquigarrow$ Feedforward-Network (FNN)

General ANN uniquely defined by **structure**:

- ▶ number m of layers
- ▶ number of neurons per layer, uniquely described by $\mathbf{n} \in \mathbb{N}^m$

Parameters for $i = 1, \dots, m - 1$:

- ▶ weight matrices $\mathbf{W}_i \in \mathbb{R}^{n_{i+1} \times n_i}$
- ▶ bias vectors $\mathbf{b}_i \in \mathbb{R}^{n_{i+1}}$
- ▶ activation functions a_i

General ANN $\rightsquigarrow$ Feedforward-Network (FNN)

General ANN uniquely defined by **structure**:

- ▶ number m of layers
- ▶ number of neurons per layer, uniquely described by $\mathbf{n} \in \mathbb{N}^m$

Parameters for $i = 1, \dots, m-1$:

- ▶ weight matrices $\mathbf{W}_i \in \mathbb{R}^{n_{i+1} \times n_i}$
- ▶ bias vectors $\mathbf{b}_i \in \mathbb{R}^{n_{i+1}}$
- ▶ activation functions a_i

“Feedforward” since evaluation of $\mathbf{f} : \mathbb{R}^{n_1} \rightarrow \mathbb{R}^{n_m}$ **layerwise forward** through network:

$$\mathbf{a}_1 := \mathbf{x}, \quad \mathbf{z}_i := \mathbf{W}_i \mathbf{a}_i + \mathbf{b}_i, \quad \mathbf{a}_{i+1} := a_i(\mathbf{z}_i), \quad i = 1, \dots, m-1, \quad \mathbf{f}(\mathbf{x}) := \mathbf{a}_m.$$

General ANN $\rightsquigarrow$ Feedforward-Network (FNN)

General ANN uniquely defined by **structure**:

- ▶ number m of layers
- ▶ number of neurons per layer, uniquely described by $\mathbf{n} \in \mathbb{N}^m$

Parameters for $i = 1, \dots, m - 1$:

- ▶ weight matrices $\mathbf{W}_i \in \mathbb{R}^{n_{i+1} \times n_i}$
- ▶ bias vectors $\mathbf{b}_i \in \mathbb{R}^{n_{i+1}}$
- ▶ activation functions a_i

“Feedforward” since evaluation of $\mathbf{f} : \mathbb{R}^{n_1} \rightarrow \mathbb{R}^{n_m}$ **layerwise forward** through network:

$$\mathbf{a}_1 := \mathbf{x}, \quad \mathbf{z}_i := \mathbf{W}_i \mathbf{a}_i + \mathbf{b}_i, \quad \mathbf{a}_{i+1} := a_i(\mathbf{z}_i), \quad i = 1, \dots, m - 1, \quad \mathbf{f}(\mathbf{x}) := \mathbf{a}_m.$$

Exercises: implementation of class `FeedforwardNet()` in Python.

Other types of ANN

In FNN every neuron in layer i connected to every neuron in layer $i + 1 \rightsquigarrow$ “fully connected layer”.

Other types of ANN

In FNN every neuron in layer i connected to every neuron in layer $i + 1 \rightsquigarrow$ “fully connected layer”.

Other ANN:

- ▶ not every neuron of a layer connected to every neuron of the next layer: specially structured matrices $\mathbf{W}_i$ (convolutional neural networks, CNN)
- ▶ connections within a layer (recurrent neural networks, RNN; amongst others LSTM and GRU)
- ▶ connections across multiple layers (residual neural networks, ResNN)
- ▶ continuous networks $\rightsquigarrow$ neural ODE

Other types of ANN

In FNN every neuron in layer i connected to every neuron in layer $i + 1 \rightsquigarrow$ “fully connected layer”.

Other ANN:

- ▶ not every neuron of a layer connected to every neuron of the next layer: specially structured matrices $\mathbf{W}_i$ (convolutional neural networks, CNN)
- ▶ connections within a layer (recurrent neural networks, RNN; amongst others LSTM and GRU)
- ▶ connections across multiple layers (residual neural networks, ResNN)
- ▶ continuous networks $\rightsquigarrow$ neural ODE

In that order we cover these other network structures.

Numerics

Let's think **numerical mathematics** $\rightsquigarrow$ fast execution of neural networks:

- ▶ **hardware libraries**: “Basic Linear Algebra Subprograms” (BLAS), “Linear Algebra PACKage” (LAPACK)
- ▶ **programming techniques**: “Compute Unified Device Architecture” (CUDA)
- ▶ **optimally adapt to the computer**, simple building blocks like matrix times vector plus vector (BLAS Level 2) much faster than naive implementation using two loops
- ▶ **standard libraries for ML**: TensorFlow (Google), PyTorch (Facebook), JAX (Google).
- ▶ makes sense to **evaluate many arguments in parallel**, since matrix times matrix plus matrix (BLAS Level 3) is much faster than sequential evaluation (and naive implementation using three loops)

Numerics

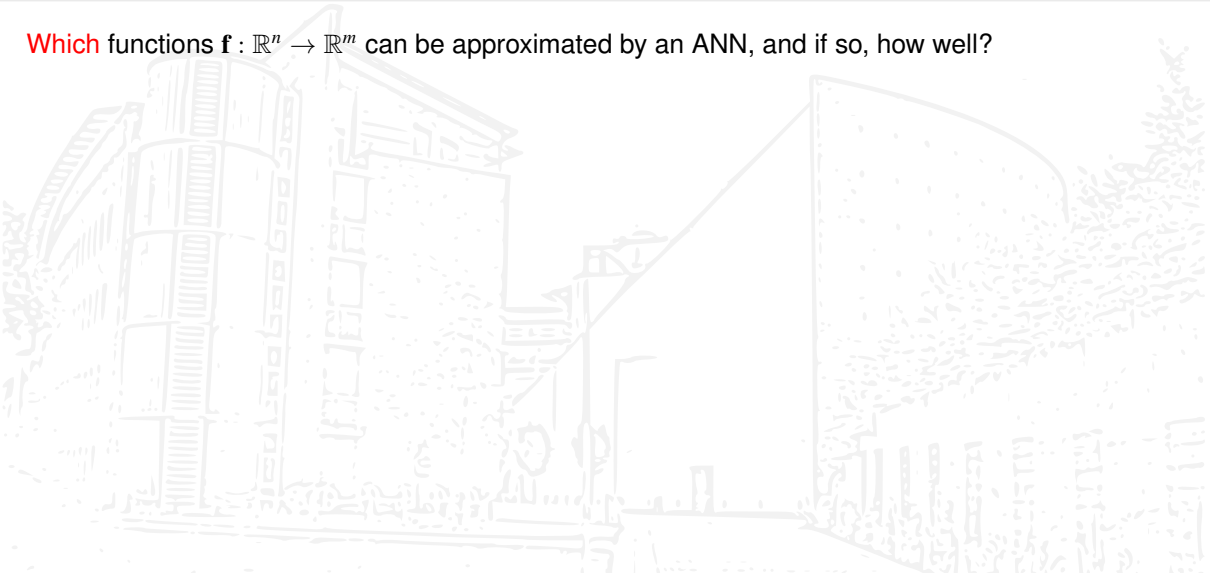
Let's think **numerical mathematics** $\rightsquigarrow$ fast execution of neural networks:

- ▶ **hardware libraries**: “Basic Linear Algebra Subprograms” (BLAS), “Linear Algebra PACKage” (LAPACK)
- ▶ **programming techniques**: “Compute Unified Device Architecture” (CUDA)
- ▶ **optimally adapt to the computer**, simple building blocks like matrix times vector plus vector (BLAS Level 2) much faster than naive implementation using two loops
- ▶ **standard libraries for ML**: TensorFlow (Google), PyTorch (Facebook), JAX (Google).
- ▶ makes sense to **evaluate many arguments in parallel**, since matrix times matrix plus matrix (BLAS Level 3) is much faster than sequential evaluation (and naive implementation using three loops)

Success of such old ideas like ANN based on today's **fast execution thanks to new hardware** (besides **availability of huge data sets for training**).

ANN are functions ... but which?

Which functions $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ can be approximated by an ANN, and if so, how well?



ANN are functions ... but which?

Which functions $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ can be approximated by an ANN, and if so, how well?

Transferred to **approximability of scalar** $f : \mathbb{R}^n \rightarrow \mathbb{R}$, preliminary work:

- ▶ Kolmogorov [3, 4]
- ▶ Sprecher [7]

ANN are functions ... but which?

Which functions $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ can be approximated by an ANN, and if so, how well?

Transferred to **approximability of scalar** $f : \mathbb{R}^n \rightarrow \mathbb{R}$, preliminary work:

- ▶ Kolmogorov [3, 4]
- ▶ Sprecher [7]

Result: every continuous function $f : [0, 1]^n \rightarrow \mathbb{R}$ of **several** variables is a finite linear combination of **univariate** functions.

ANN are functions ... but which?

Which functions $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ can be approximated by an ANN, and if so, how well?

Transferred to **approximability of scalar** $f : \mathbb{R}^n \rightarrow \mathbb{R}$, preliminary work:

- ▶ Kolmogorov [3, 4]
- ▶ Sprecher [7]

Result: every continuous function $f : [0, 1]^n \rightarrow \mathbb{R}$ of **several** variables is a finite linear combination of **univariate** functions.

Pros and Cons:

- ▶ functions **continuous**
- ▶ ANN w/ **one hidden layer** of $2n + 1$ neurons
- ▶ functions **unknown activation functions**
- ▶ functions per layer **different, not known in advance**

References I

- [1] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. URL: <http://www.deeplearningbook.org>.
- [2] Catherine F. Higham and Desmond J. Higham. “Deep learning: an introduction for applied mathematicians”. In: *SIAM Rev.* 61.4 (2019), pp. 860–891. ISSN: 0036-1445. DOI: 10.1137/18M1165748. URL: <https://doi.org/10.1137/18M1165748>.
- [3] A. N. Kolmogorov. “On the representation of continuous functions of many variables by superposition of continuous functions of one variable and addition”. In: *Dokl. Akad. Nauk SSSR* 114.5 (1957), pp. 953–956. ISSN: 0002-3264.
- [4] A. N. Kolmogorov. “On the representation of continuous functions of several variables by superpositions of continuous functions of a smaller number of variables”. In: *Amer. Math. Soc. Transl. (2)* 17 (1961), pp. 369–373. ISSN: 0065-9290. DOI: 10.1090/trans2/017/12. URL: <https://doi.org/10.1090/trans2/017/12>.
- [5] Michael A. Nielsen. *Neural Networks and Deep Learning*. Determination Press, 2015. URL: <http://neuralnetworksanddeeplearning.com>.

References II

- [6] Daniel A. Roberts and Sho Yaida. *The Principles of Deep Learning Theory: An Effective Theory Approach to Understanding Neural Networks*. Based on research in collaboration with Boris Hanin. Cambridge University Press, May 2022. ISBN: 9781009023405. DOI: [10.1017/9781009023405](https://doi.org/10.1017/9781009023405).
- [7] David A. Sprecher. “On the structure of continuous functions of several variables”. In: *Trans. Amer. Math. Soc.* 115 (1965), pp. 340–355. ISSN: 0002-9947. DOI: [10.2307/1994273](https://doi.org/10.2307/1994273). URL: <https://doi.org/10.2307/1994273>.
- [8] Aston Zhang et al. *Dive into Deep Learning*. Amazon/Online at GitHub, 2020. URL: <https://d2l.ai/>.